MODULE 4 | SESSION 15

Code Review and Cognitive Debt Mitigation

A passing test suite is not the same thing as a safe change.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

AI can ship code that looks right, sounds right, and still be structurally wrong.

- Syntax can be clean.
- Tests can still pass.
- The architecture can still drift.
- A hidden logic bug can still survive review.

Review failure mode

Polished AI output can create false confidence. The team starts approving fluency instead of understanding.

Review standard

The reviewer has to explain the change, the boundary it touches, and the risk it introduces.

Review must go beyond correctness theater.

- What changed in control flow?
- What changed in ownership or boundaries?
- Which failure modes were introduced?
- What assumption got smuggled into the diff?

```
Review the load-bearing behavior

- invariants
- data ownership
- permission boundaries
- error handling
- side effects
- integration seams
```

Passing tests are evidence, not proof.

What tests prove

The specific cases the suite currently expresses and checks.

What tests do not prove

That untouched paths, integration seams, and architectural intent remain safe.

Reviewer obligation

Rebuild the mental model and look for what the tests forgot to ask.

Practical discipline

Treat green checks as one signal, not a permission slip to stop thinking.

Automation bias and cognitive debt reinforce each other.

- Polished output feels trustworthy.
- Confident narration suppresses scrutiny.
- Reviewers accept the answer too quickly.
- The repo grows while comprehension shrinks.

Automation bias

Over-trusting the machine because it sounds certain or because prior outputs were good enough.

Cognitive debt

Approving code the team does not deeply understand and leaving future engineers to carry the cost.

A strong review checklist looks for structural risk, not just clean formatting.

Boundaries

Does the change preserve ownership, interfaces, and trust assumptions?

Error handling

Do failure paths stay explicit, recoverable, and observable?

Logic drift

Did a small diff quietly change business meaning or control flow?

Maintainability

Will the next reviewer understand the system more clearly or less clearly after this lands?

The demo should end with an actionable review comment, not just a vague concern.

Step 1

Present a short AI-generated PR with a subtle logic or architectural flaw.

Step 2

Walk the diff line by line until the hidden assumption becomes visible.

Step 3

Write a review comment that names the issue, impact, and smallest useful fix.

> If the reviewer cannot explain the diff later, the review did not finish.

```
Good review output

- identify the risk
- explain the impact
- point to the boundary
- ask for the smallest fix
- leave evidence for later readers
```

Good review is understanding plus evidence.

- Green tests are not the finish line.
- Review has to examine the load-bearing behavior.
- If the reviewer cannot explain the change, the review is incomplete.

Bridge to Session 16

Session 15 strengthens human judgment. Session 16 makes that judgment traceable, reconstructable, and auditable later.